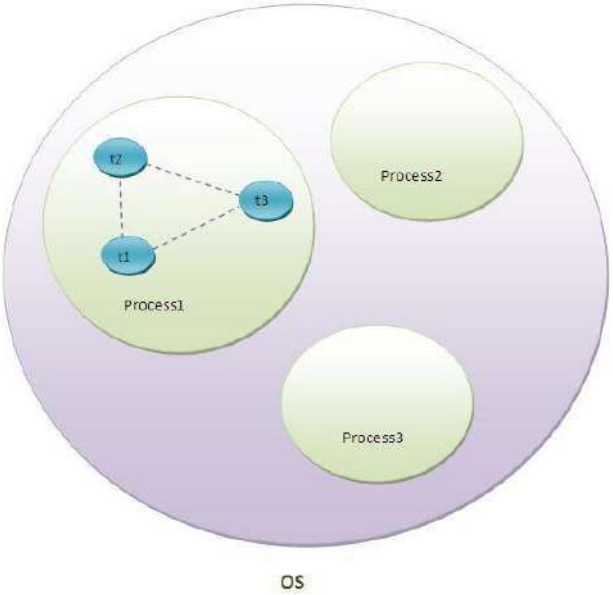
Unit 4. Threads and Packages

4.1 What is a Thread in Java?



A **thread** in Java is a lightweight sub process, the smallest unit of processing. It is a separate path of execution within a program. Threads are independent of each other; if an exception occurs in one thread, it does not affect other threads. They share a common memory area, enabling efficient communication between them.

As shown in the figure, a thread is executed inside a process. There is **context-switching** between threads, meaning that the CPU can switch between different threads. A single process can have multiple threads, and there can be multiple processes within an operating system.

Multitasking

Multitasking is the process of executing multiple tasks simultaneously to utilize the CPU effectively. Multitasking can be achieved in two ways:

1. **Process-based Multitasking (Multiprocessing)**:
   - Each process has its own memory address space (separate memory allocation).
   - Processes are considered heavyweight.
   - Communication between processes is costly in terms of performance.
   - Switching from one process to another requires significant overhead (saving/loading registers, memory maps, etc.).
2. **Thread-based Multitasking (Multithreading)**:
   - Threads share the same address space, making them lightweight.

- Communication between threads is more efficient and less costly.

Multithreading in Java

Multithreading in Java is a process of executing multiple threads simultaneously. A thread is a lightweight sub-process and the smallest unit of processing. While **multiprocessing** and **multithreading** both aim to achieve multitasking, multithreading is often preferred because threads share a common memory area, saving memory and offering faster context-switching compared to processes.

Java multithreading is widely used in applications like games, animations, and real-time simulations.

Advantages of Java Multithreading

1. **Non-blocking User Interface**: Since threads are independent, multiple operations can run simultaneously, ensuring that the user interface remains responsive.
2. **Faster Execution**: Multiple operations can be performed at once, which saves time.
3. **Isolation of Threads**: Since threads are independent, an exception in one thread does not affect others.

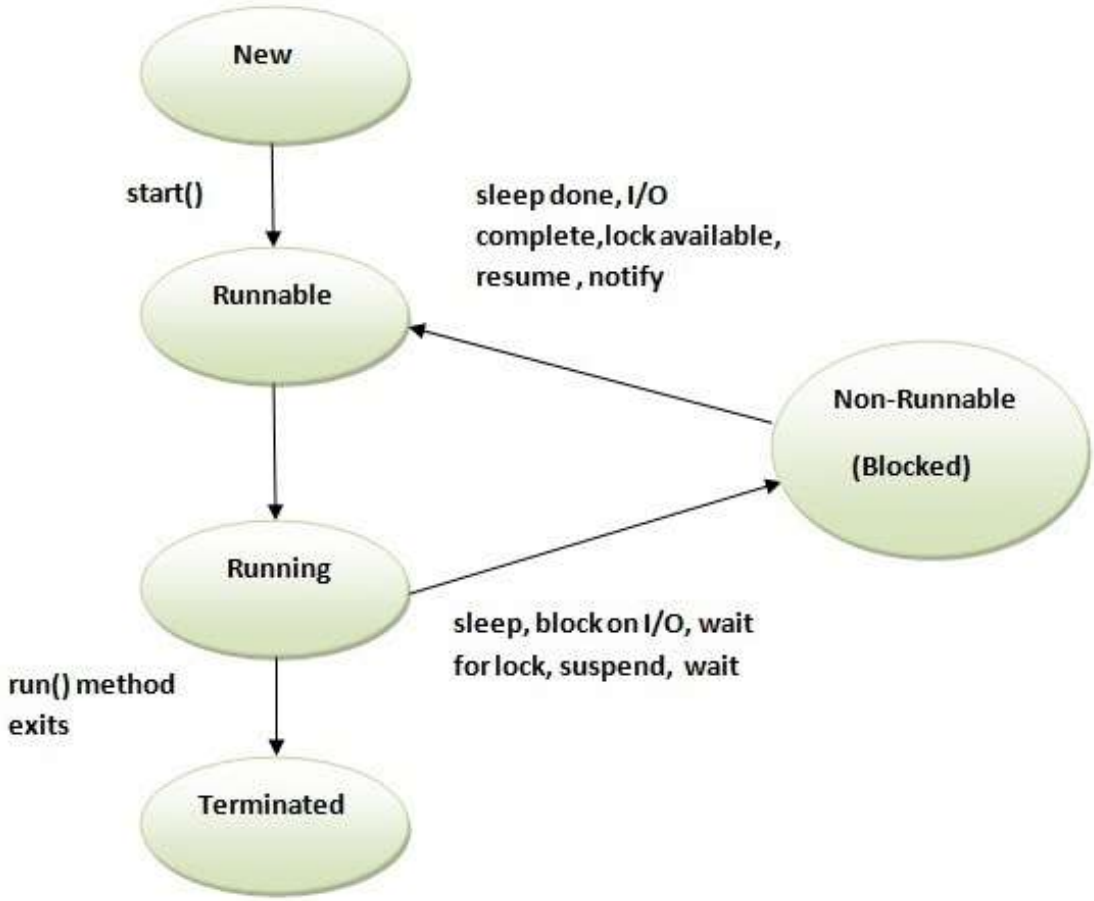
4.1.2 Life Cycle of a Thread (Thread States)

In Java, a **thread** goes through several states during its life cycle, controlled by the **Java Virtual Machine (JVM)**. These states are used to represent the various stages a thread can be in while it is executing or waiting for resources.

A thread can be in one of the following **five states**:

1. **New**: A thread is in the *New* state when an instance of the Thread class is created but before the start() method is invoked.
2. **Runnable**: A thread is in the *Runnable* state after the start() method is invoked but before it is selected by the thread scheduler.
3. **Running**: A thread is in the *Running* state when the thread scheduler selects it for execution.
4. **Non-Runnable (Blocked)**: A thread is alive but not eligible to run, for example, when it is waiting for I/O operations.

5. **Terminated**: A thread enters the *Terminated* (or *Dead*) state when its run() method finishes executing.



Methods for Controlling Thread States

- **start()**: Transitions a thread from **New** to **Runnable**.
- **run()**: Contains the code that is executed when the thread is **Running**.
- **sleep(long milliseconds)**: Causes the thread to sleep (pause execution) for the specified time, moving the thread into **Blocked** state temporarily.
- **join()**: Allows one thread to wait for another thread to complete its execution, typically used for synchronization, moving the thread into a **Blocked** state.
- **interrupt()**: Can be used to interrupt a thread, causing it to stop its execution or move to the **Terminated** state.

Commonly used Constructors of Thread class:

- Thread()
- Thread(String name)
- Thread(Runnable r)
- Thread(Runnable r, String name)

Commonly used methods of Thread class:

1. **public void run():** is used to perform action for a thread.
2. **public void start():** starts the execution of the thread.JVM calls the run() method on the thread.
3. **public void sleep(long miliseconds):** Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.
4. **public void join():** waits for a thread to die.
5. **public void join(long miliseconds):** waits for a thread to die for the specified miliseconds.
6. **public int getPriority():** returns the priority of the thread.
7. **public int setPriority(int priority):** changes the priority of the thread.
8. **public String getName():** returns the name of the thread.
9. **public void setName(String name):** changes the name of the thread.
10. **public boolean isAlive():** tests if the thread is alive.

How to Create a Thread in Java

There are two ways to create a thread in Java:

1. **By extending Thread classs**
2. **By implementing Runnable interface.**

1. **By Extending the Thread Class**:
    - The Thread class provides constructors and methods to create and perform operations on a thread.
    - The Thread class can also implement the Runnable interface.

Example:

```
import java.io.*;
import java.util.*;

public class EXThreadByClass extends Thread {
```

```
    // Define the run method for the thread
    public void run() {
        System.out.println("Thread Started Running...");
    }
    public static void main(String[] args) {
        EXThreadByClass g1 = new EXThreadByClass();
        // Start the thread using the start() method
        g1.start();
    }
}
```

Output:

Thread Started Running...

2. **By Implementing the Runnable Interface**:
    - The Runnable interface is implemented by a class whose instances are executed by a thread. You do not need to subclass the Thread class if you implement Runnable.

Steps:

- Implement the run() method.
- Instantiate the Thread class and pass the Runnable implementer to it.
- Call the start() method to begin the thread execution.

Example:

```
import java.io.*;
import java.util.*;

public class EXThreadByInterface implements Runnable
{
    // Define the run method for the thread
    public void run()
    {
        System.out.println("Thread is Running Successfully");
    }
    public static void main(String[] args)
   {
```

```
    EXThreadByInterface g1 = new EXThreadByInterface();
    // Create a new thread and pass the Runnable implementer to it
    Thread t1 = new Thread(g1);
    t1.start();
  }
}
```

Output:

Thread is Running Successfully

Thread methods with example:

1. sleep()
2. isAlive()

1. sleep()

- The **sleep( )** method causes the thread from which it is called to suspend execution for the specified period of milliseconds.
- It is one method of the Runnable class.
- The pause of threads is accomplished with this method.
- Its general form is shown here:

static void sleep(long *milliseconds*) throws InterruptedException

- The number of milliseconds to suspend is specified in *milliseconds.*
- This method may throw an **InterruptedException**.
- The **sleep( )** method has a second form, which allows you to specify the period in terms of milliseconds and nanoseconds:

static void sleep(long *milliseconds,* int *nanoseconds*) throws InterruptedException

- This second form is useful only in environments that allow timing periods as short as nanoseconds.

Example:

```
class CurrentThreadDemo
{
    public static void main(String args[])
    {
            Thread t=Thread.currentThread();
            System.out.println("current thread:"+t);
```

```
            t.setName("my thread");
            System.out.println("after name change:="+t);
            try
            {
                for(int n=5;n>0;n++)
                {
                    System.put.println(n);
                    Thread.sleep(1000);
                }
            }
            catch(InterrupedException e)
            {
                System.out.println("main thread interrupted");
            }
        }
    }
```

2. isAlive()

- By calling **isAlive()** method on the thread we can determine whether a thread has finished or not.
- This method is defined by **Thread**, and its general form is shown here:

  final boolean isAlive( )
- The **isAlive( )** method returns **true** if the thread upon which it is called is still running. It returns **false** otherwise.
- While **isAlive( )** is occasionally useful, the method that you will more commonly use to wait for a thread to finish is called **join( )**,

Example:

```
// Using join() to wait for threads to finish.
class NewThread implements Runnable
{
    String name; // name of thread
    Thread t;
    NewThread(String threadname)
    {
        name = threadname;
```

```
        t = new Thread(this, name);
        System.out.println("New thread: " + t);
        t.start(); // Start the thread
    }
    // This is the entry point for thread.
    public void run()
    {
        try
        {
                for(int i = 5; i > 0; i—)
                {
                System.out.println(name + ": " + i);
                Thread.sleep(1000);
                }
        }
        catch (InterruptedException e)
        {
                System.out.println(name + " interrupted.");
        }
        System.out.println(name + " exiting.");
    }
}

class DemoJoin
{
public static void main(String args[])
        {
        NewThread ob1 = new NewThread("One");
        NewThread ob2 = new NewThread("Two");
        NewThread ob3 = new NewThread("Three");

        System.out.println("Thread One is alive: " + ob1.t.isAlive());
        System.out.println("Thread Two is alive: " + ob2.t.isAlive());
        System.out.println("Thread Three is alive: "+ ob3.t.isAlive());

        // wait for threads to finish
        try
        {
```

```
            System.out.println("Waiting for threads to finish.");
            ob1.t.join();
            ob2.t.join();
            ob3.t.join();
            }
            catch (InterruptedException e)
            {
            System.out.println("Main thread Interrupted");
            }
            System.out.println("Thread One is alive: " + ob1.t.isAlive());
            System.out.println("Thread Two is alive: " + ob2.t.isAlive());
            System.out.println("Thread Three is alive: " + ob3.t.isAlive());
            System.out.println("Main thread exiting.");
        }
}
```

4.2 Thread Priority

Each thread has a **priority**, represented by an integer between 1 and 10. The thread scheduler may use the priority to decide the order in which threads are executed (known as **preemptive scheduling**), but this is not guaranteed. The Java Thread class defines three constants for thread priority:

1. **MIN_PRIORITY** (1)
2. **NORM_PRIORITY** (5)
3. **MAX_PRIORITY** (10)

By default, threads are assigned a priority of NORM_PRIORITY (5).

Example:

```
class TestMultiPriority1 extends Thread
{
   public void run()
    {
      System.out.println("Running thread name is: " + Thread.currentThread().getName());
      System.out.println("Running thread priority is: " + Thread.currentThread().getPriority());
   }

   public static void main(String args[])
```

```
  {
    TestMultiPriority1 m1 = new TestMultiPriority1();
    TestMultiPriority1 m2 = new TestMultiPriority1();
    m1.setPriority(Thread.MIN_PRIORITY);
    m2.setPriority(Thread.MAX_PRIORITY);
    m1.start();
    m2.start();
  }
}
```

Output:

```
Running thread name is: Thread-0
Running thread priority is: 10
Running thread name is: Thread-1
Running thread priority is: 1
```

4.2 Package Naming, Type Imports

4.2.1 Package Access, Package contents

Packages in Java

- A **package** is essentially a group of classes.
- Packages act as containers for classes, helping to keep the class namespace compartmentalized.
- For example, a package allows you to create a class named List, which can be stored in your own package without concern that it will collide with another class named List stored elsewhere.

Package as a Naming and Visibility Control Mechanism

- A package serves both as a **naming mechanism** and a **visibility control mechanism**.
- You can define classes inside a package that are not accessible by code outside that package.
- You can also define class members that are only accessible to other members of the same package.

Defining a Package

1. To create a package, include a package statement as the first statement in a Java source file.
2. Any classes declared within that file will belong to the specified package.
3. The package statement defines a namespace in which classes are stored. If you don't specify a package name, the class names will be put into the **default package**, which has no name.

Syntax of the Package Statement:

package packagename;

- Here, packagename is the name of the package.

For example, the following statement creates a package called MyPackage:

package MyPackage;

File System and Package Storage

- Java uses file system directories to store packages.
- For example, the .class files for any classes declared to be part of MyPackage must be stored in a directory named MyPackage. The directory name must match the package name exactly, and case is significant.
- By default, the Java runtime system uses the **current working directory** as its starting point.

Characteristics of Packages

1. **Hierarchical Structure**: Packages are stored in a hierarchical manner and are explicitly imported into new class definitions.
2. **Naming and Visibility Control**: The package serves both as a naming and visibility control mechanism.
3. **Default Package**: If a source file does not contain a package statement, its classes and interfaces are placed in a default package.
4. **Package Statement**: The package statement must appear as the first statement in the source file.
5. **Class Name Conflict**: Two different packages can declare classes with the same name. This allows Java to partition the class namespace into more manageable chunks.
6. **Access Control**: Classes inside a package can be defined to not be accessible from outside the package. Similarly, class members can be exposed only to other members of the same package.

Benefits of Organizing Classes into Packages

1. **Reusability**: Classes contained in packages can easily be reused by other programs.
2. **Unique Classes**: In packages, classes can have the same name as those in other packages. This prevents name conflicts.
3. **Encapsulation**: Packages can hide classes meant for internal use only, preventing external access.
4. **Separation of Design and Code**: Packages provide a way to separate the design from the implementation. Initially, you can design the classes and decide their responsibilities, and then implement the required methods without affecting the overall design.

Hierarchy of Packages

- You can create a hierarchy of packages by separating each package name from the one above it using a period (.).

  Syntax:
  package pkg1[.pkg2[.pkg3]];

  In this case:

  - pkg1 is the top-level package.
  - pkg2 is a sub-package of pkg1.
  - pkg3 is a sub-package of pkg2.

By organizing classes into packages, you achieve a well-structured and maintainable codebase, which improves the clarity, reuse, and encapsulation of your code.

How To create package

- To create a package, include a **package** command as the first statement in a Java source file.
- Any classes declared within that file will belong to the specified package.

- The **package** statement defines a name space in which classes are stored.

-If you will not give the package name then the class names are put into the default package, which has no name.

1. This is the general form or syntax of the **package** statement:
    1. **package *packagename*;**
2. Here, *packagename* is the name of the package.
3. For example, the following statement creates a package called **MyPackage**.
    1. **package MyPackage;**

How to create a package

Suppose we have a file called HelloWorld.java, and we want to put this file in a package **world**. First thing we have to do is to specify the keyword **package** with the name of the package we want to use (**world**in our case) on top of our source file, before the code that defines the real classes in the package, as shown in our HelloWorld class below:

```
// only comment can be here
package world;

public class HelloWorld {
  public static void main(String[] args) {
    System.out.println("Hello World");
  }
}
```

Setting up the CLASSPATH

set CLASSPATH=.;C:\;

We set the CLASSPATH to point to 2 places, . (dot) and C:\ directory.

Compile package

When compiling HelloWorld class, we just go to the world directory and type the command:

C:\world\javac HelloWorld.java

If we want to run it, we have to tell JVM about its **fully-qualified class name** (world.HelloWorld) instead of its plain class name(HelloWorld).

Run Package

C:\world>java world.HelloWorld

C:\world>Hello World
Note: **fully-qualified class name** is the name of the java class that includes its package name

4.2.2 Package Access Specification

- As you will see, Java provides many levels of protection to allow fine-grained control over the visibility of variables and methods within classes, subclasses, and packages.

- Packages act as containers for classes and other subordinate packages.

- Java addresses four categories of visibility for class members:
  - Subclasses in the same package
  - Non-subclasses in the same package
  - Subclasses in different packages
  - Classes that are neither in the same package nor subclasses

- The three access specifiers, **private**, **public**, and **protected**, provide a variety of ways to produce the many levels of access required by these categories.

CLASS MEMBER ACCESS

	PRIVATE	NO MODIFIER	PROTECTED	PUBLIC
SAME CLASS	YES	YES	YES	YES
SAME PACKAGE SUBCLASS	NO	YES	YES	YES
SAME PACKAGE NON-SUBCLASS	NO	YES	YES	YES
DIFFERENT PACKAGE SUBCLASS	NO	NO	YES	YES
DIFFERENT PACKAGE NON-SUBCLASS	NO	NO	NO	YES

1] PUBLIC

- A class has only two possible access levels: default and public.
- When a class is declared as **public**, it is accessible by any other code.

2] DEFAULT [friendly access]

4. If a class has default access, then it can only be accessed by other code within its same package.
5. The difference between public access and friendly access is that the public modifier makes fields visible in all classes regardless of their packages, while friendly access modifier makes field's visibility only in the same package but not in other packages.

3] PRIVATE

- Anything declared **private** cannot be seen outside of its class.
- Can't be inherited by the subclasses of other packages.

4] PROTECTED

Protected modifier makes the field visible not only to all classes and subclasses in the same package as well as in other packages.

5] PRIVATE PROTECTED

This modifier makes the fields visible in all sub classes regardless of what package they are in. These fields are not accessible by other classes in the same package.

EXAMPLE

The following example shows all combinations of the access control modifiers.

Following is the source code for the other package, **p1**.

This is file **Protection.java**:

```
package p1;
public class Protection
{       int n = 1;
        private int n_pri = 2;
        protected int n_pro = 3;
        public int n_pub = 4;
        public Protection()
        {       System.out.println("base constructor");
                System.out.println("n = " + n);
                System.out.println("n_pri = " + n_pri);
                System.out.println("n_pro = " + n_pro);
                System.out.println("n_pub = " + n_pub);
        }
}
```

This is file **Derived.java**:

```
package p1;
class Derived extends Protection
{       Derived()
         {      System.out.println("derived constructor");
                System.out.println("n = " + n);
                // class only
                // System.out.println("n_pri = " + n_pri);
                System.out.println("n_pro = " + n_pro);
                System.out.println("n_pub = " + n_pub);
        }
}
```

This is file **SamePackage.java**:

```
package p1;
class SamePackage
{       SamePackage()
         {      Protection p = new Protection();
                System.out.println("same package constructor");
                System.out.println("n = " + p.n);
                // class only
```

```
                // System.out.println("n_pri = " + p.n_pri);
                System.out.println("n_pro = " + p.n_pro);
                System.out.println("n_pub = " + p.n_pub);
        }
}
```

Following is the source code for the other package, **p2**.

This is file **Protection2.java**:

```
package p2;
class Protection2 extends p1.Protection
{       Protection2()
        {       System.out.println("derived other package constructor");
                // class or package only
                // System.out.println("n = " + n);
                // class only
                // System.out.println("n_pri = " + n_pri);
                System.out.println("n_pro = " + n_pro);
                System.out.println("n_pub = " + n_pub);
        }
}
```

This is file **OtherPackage.java**:

```
package p2;
class OtherPackage
{       OtherPackage()
        {       p1.Protection p = new p1.Protection();
                System.out.println("other package constructor");
                // class or package only
                // System.out.println("n = " + p.n);
                // class only
                // System.out.println("n_pri = " + p.n_pri);
                // class, subclass or package only
                // System.out.println("n_pro = " + p.n_pro);
                System.out.println("n_pub = " + p.n_pub);
        }
}
```

Here are two test files you can use. The one for package p1 is shown here:

```
        // Demo package p1.
        package p1;
        // Instantiate the various classes in p1.
        public class Demo
        {       public static void main(String args[])
                {       Protection ob1 = new Protection();
                        Derived ob2 = new Derived();
                        SamePackage ob3 = new SamePackage();
                }
```

```
}
```

The test file for p2 is shown next:

```
// Demo package p2.
package p2;
// Instantiate the various classes in p2.
public class Demo
 {      public static void main(String args[])
        {       Protection2 ob1 = new Protection2();
                OtherPackage ob2 = new OtherPackage();
        }
}
```